

A Hard-to-Beat Rule for Week-Ahead Traffic Forecasting

The Calendar-Adjusted Seasonal Baseline

Traffic Forecasting Project

June 2026

Abstract

We describe the best one-week-ahead forecasting rule we found for California highway traffic *flow*, on the Greater Bay Area subset of the LargeST dataset (2,352 sensors, hourly, 2017–2019). It is deliberately simple: a recency-weighted average of the same hour-of-week over recent weeks, multiplied by a learned holiday/calendar factor on special days. On a rolling 2019 backtest it reaches $R^2 = 0.949$ and a weekly-aggregate MAPE of 3.55%, and we could not beat it – not with autoregression, gradient-boosted trees, pooled panel models, or cross-regional information. This report states the rule precisely, works through small numerical examples, explains *why* it is so hard to beat, and closes with concrete thoughts on where genuine improvement might still come from.

1 The rule

Let y_t be the traffic flow (vehicles per hour) at a given sensor in clock-hour t . Traffic is strongly periodic with a one-week period: the same weekday-and-hour looks very similar week to week (Figure 1). It is convenient to index by the *hour-of-week*, $\phi(t) = t \bmod 168 \in \{0, \dots, 167\}$, which encodes both the day of week and the hour of day.

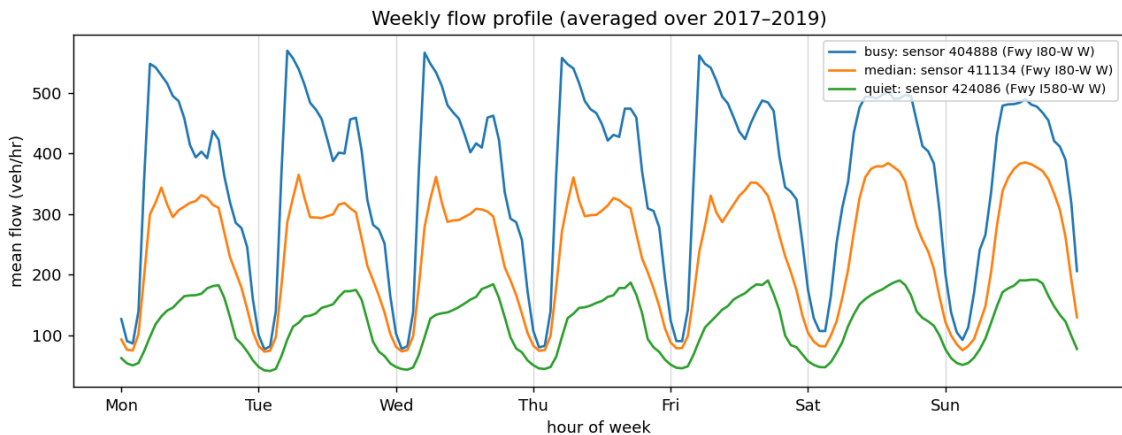


Figure 1: Mean flow by hour-of-week for three sensors. The weekday commute double-peak and the weekend single hump repeat with high regularity – the deterministic “shape” the rule exploits.

1.1 Step 1 – the seasonal baseline (last-4-weeks average)

Predict each future hour by its recent average *at the same hour of week*:

$$b_t = \frac{y_{t-168} + y_{t-336} + y_{t-504} + y_{t-672}}{4}, \quad (1)$$

that is, the mean of the same weekday-and-hour over the previous four weeks (the lags 168, 336, 504, 672 are simply 1, 2, 3, 4 weeks in hours). Every term is at least a week old, so at the forecast origin the *entire* next week is computable from data already in hand – the rule is leakage-free for week-ahead use.

Worked example. Forecast next **Wednesday 08:00** (a commute peak) at a busy sensor. The last four Wednesdays at 08:00 read 540, 560, 535, 555 veh/h (oldest to newest). Then

$$b = \frac{1}{4}(540 + 560 + 535 + 555) = \mathbf{547.5} \text{ veh/h.}$$

1.2 Step 2 – recency weighting (a one-parameter upgrade)

When the level is drifting (slow growth, a seasonal transition), weight recent weeks more. Use geometric weights $\propto \alpha^0, \alpha^1, \alpha^2, \alpha^3$ with $\alpha \approx 0.8$, normalized to sum to one – about (0.34, 0.27, 0.22, 0.17) from the most recent week to the oldest:

$$b_t^{\text{ewma}} = 0.34 y_{t-168} + 0.27 y_{t-336} + 0.22 y_{t-504} + 0.17 y_{t-672}. \quad (2)$$

Worked example (rising level). If the last four same-hour values are 500, 520, 540, 560 veh/h (oldest to newest), the flat average is 530, while the recency-weighted value is

$$0.34(560) + 0.27(540) + 0.22(520) + 0.17(500) = \mathbf{537.4} \text{ veh/h,}$$

leaning toward the recent, higher values and so tracking the upward drift the flat average lags. (In a flat period the two coincide.) Fitting the single decay α on past data is the only “training” in the whole rule, and a fixed $\alpha = 0.8$ works well.

1.3 Step 3 – the holiday / calendar adjustment

The baseline has one clear blind spot: **holidays**. To forecast the Fourth of July it averages four *ordinary* weeks and badly over-predicts, because a holiday reshapes the day – the morning commute largely disappears (Figure 2). Holidays are only about 8% of hours but contribute roughly 23% of the squared error, and they are *known in advance* – so we correct them explicitly with a multiplicative factor.

For a given holiday and a given hour of the day, define the factor in words:

the factor is the average, over past occurrences of that holiday, of the ratio (actual flow) / (baseline flow) at that hour of the day.

Concretely, if a holiday has occurred twice in the training data and at 17:00 its flow was 0.70 and 0.72 times the baseline on those two days, the factor is

$$f = \frac{0.70 + 0.72}{2} = 0.71. \quad (3)$$

A factor below one means the holiday-hour runs below its ordinary-week level; the adjusted forecast on special days is simply

$$\hat{y}_t = b_t^{\text{ewma}} \times f. \quad (4)$$

Worked example. Suppose the baseline for the Fourth of July at 17:00 is 490 veh/h (from the four ordinary weeks before it) and the learned factor for “Independence Day, 17:00” is 0.71. Then

$$\hat{y} = 490 \times 0.71 = \mathbf{348 \text{ veh/h}},$$

a 29% correction exactly where the plain baseline fails.

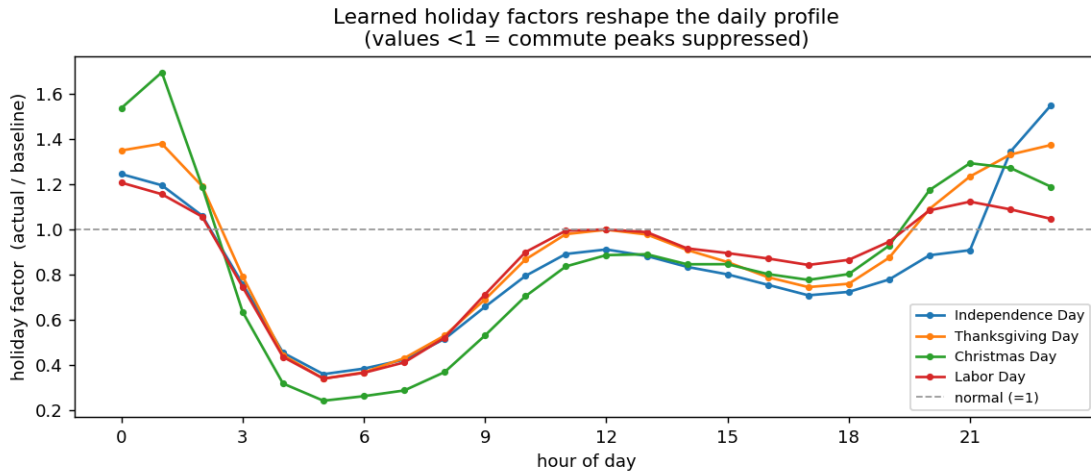


Figure 2: Learned holiday factors by hour of day. Values below one crush the commute peaks (mornings drop to ~ 0.35 – 0.45); the late-evening values above one reflect holiday-eve travel.

Practicalities that matter.

- **Multiplicative, not additive:** the holiday effect scales with the level, so a ratio transfers across busy and quiet weeks; a fixed offset does not.
- **Per hour of day:** holidays reshape the daily profile, so the factor varies across the 24 hours (Figure 2).
- **Pool across years** (and across sensors) to denoise the factor – each holiday occurs only once a year.
- **Include shoulder days:** the day before and after a holiday, and “bridge” days, each get their own factor.
- **Known-ahead:** because holidays come from the calendar, the factor is available at any horizon – no forecast of an input is required.

1.4 The rule, in one line

Forecast = (recency-weighted average of the same hour-of-week over the last few weeks)
 $\times$ (holiday factor on special days, otherwise 1).

A worked holiday day, hour by hour, shows the two pieces combining:

Table 1: A holiday reshaped by the learned factor (illustrative values).

hour	baseline b^{ewma} (veh/h)	factor f	forecast $\hat{y}$ (veh/h)
08:00	520	0.42	218
12:00	430	0.96	413
17:00	490	0.71	348
22:00	240	1.30	312

2 Why it is hard to beat

Performance. On a rolling-origin 2019 backtest (factors learned only from 2017–18), the pieces stack cleanly:

Table 2: One-week-ahead accuracy, GBA flow, test 2019 (all 2,352 sensors).

Model	R^2	RMSE (veh/h)	RMSE on holiday hours
Flat 4-week baseline	0.9404	40.14	66.96
+ recency weighting (EWMA)	0.9435	39.10	65.99
+ holiday factor	0.9493	37.01	47.71

The holiday factor cuts holiday-hour error by 29% ($67 \rightarrow 48$ veh/h) and leaves ordinary hours untouched; the weekly-aggregate MAPE is 3.55%.

It sits at the practical ceiling. Fit the best possible structure *in-sample* on 2019 itself (perfect parameters, an upper bound): a static hour-of-week mean reaches $R^2 = 0.938$, and adding an oracle holiday effect 0.943. Our *out-of-sample* rule scores 0.949 – *above* the in-sample static oracle, because the recency weighting tracks within-year drift a fixed mean cannot. The remaining $\sim 5\%$ of variance is dominated by irreducible, incident-driven noise that no week-ahead model can recover.

Everything fancier failed to beat it. We tried the usual escalations; none added meaningful skill (Table 3). The reason is structural: the hour-of-week shape is deterministic and already captured, what little autoregressive signal exists in the residual decays away over a one-week horizon, and calendar/sensor features the baseline already encodes carry no new information.

Table 3: Attempts to beat the rule on endogenous information, and what they added.

Attempt	Effect vs. the rule
Autoregression on the residual (hourly lags)	$\leq +0.003 R^2$; decays to 0 by a week out
Autoregression on weekly-aggregate residual	$\leq +0.001 R^2$
Ridge / gradient boosting on calendar + sensor features	≈ 0 (high-cardinality features overfit)
Pooled panel AR (coefficients shared, +Fourier)	worse than the rule
Global gradient boosting on the weekly panel	worse than the rule
Cross-regional common factor (LA, San Diego)	exists but is unforecastable (white in time)

The through-line: on this problem, **a one- or two-parameter rule is at the ceiling**, because the predictable part is a deterministic calendar pattern and the rest is noise. Model complexity has nothing to feed on.

3 Thoughts on how to improve it

We do not think more sophisticated models of the *same information* will help. Genuine gains have to come from new information or a changed objective.

1. **Known-ahead exogenous signals.** The holiday factor works because holidays are calendar-known anomalies the baseline misses. The natural generalization is other *known-ahead* events: scheduled sport/concerts and planned lane closures / roadwork. These are valid at any horizon (no forecast of the input needed). The caveat is granularity – a stadium event is large but localized and brief, so it helps the few sensors near the venue at the day/hour level, not the network aggregate.
2. **Longer horizons and a longer record.** One week ahead is essentially solved; the open problem is multi-week, where a slowly-drifting *level* governs the error. Modeling that level (trend, annual cycle) becomes worthwhile from about three to four weeks out, and learning slow multi-year structure needs more than three years of data.
3. **Distributional forecasts.** Everything above is a point forecast, but much operational value is in the tails – the probability of severe congestion, calibrated intervals. The residual is heteroscedastic (heavy-tailed around peaks and incidents), so quantile or distributional methods add value for *risk*, not for the mean.
4. **Harder targets.** We forecast flow, which is highly seasonal. Speed is far noisier (a parallel check reached only $R^2 \approx 0.76$) and is a genuinely harder, more open problem.

The unifying lesson is simple: **complexity should follow information and objective, not precede them.** For week-ahead flow, the information is a clean calendar pattern and the objective is the mean – so a calendar-adjusted seasonal average is not just a baseline, it is close to the best one can do.

Reproducibility. All numbers and figures come from the project’s `src/` scripts (the seasonal/EWMA/holiday models, the oracle-floor and feature-merit analyses, and the pooled and cross-regional experiments), on the open LargeST dataset.